

Python Control Flow

Conditionals, Comparison & Logical Operators, Loops, and Loop Control

1. Introduction

Control flow is what allows a program to make decisions and repeat actions instead of executing every statement exactly once from top to bottom. Without control flow, programs would be limited to simple, linear sequences of instructions. This module covers Python's conditional statements (`if`, `elif`, `else`), the comparison and logical operators used to build conditions, and the two looping constructs — `for` and `while` — along with the `range()` function, loop control statements (`break` and `continue`), and nested loops.

Mastering control flow is essential because nearly every non-trivial program needs to branch based on conditions (e.g. 'if the user is logged in, show the dashboard') or repeat work over a collection of data (e.g. 'for every item in the shopping cart, add its price to the total').

2. Learning Objectives

- Write conditional branches using `if`, `elif`, and `else`.
- Combine conditions using comparison operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) and logical operators (`and`, `or`, `not`).
- Write `for` loops to iterate over sequences and ranges of numbers.
- Write `while` loops that repeat based on a condition, including loops that run indefinitely until stopped.
- Use `range()` with one, two, and three arguments to generate number sequences.
- Control loop execution using `break` and `continue`.
- Build and reason about nested loops.

3. Concept Explanations

3.1 Conditional Statements: `if` / `elif` / `else`

An `if` statement executes a block of code only when its condition evaluates to `True`. `elif` (short for 'else if') lets you check additional conditions in sequence, and `else` provides a fallback block that runs when none of the previous conditions were true. Python evaluates conditions top to bottom and executes only the first block whose condition is true.

```
temperature = 28
```

```
if temperature > 35:
    print("It's very hot")
elif temperature > 25:
    print("It's warm")
elif temperature > 15:
```

```

    print("It's mild")
else:
    print("It's cold")

```

3.2 Comparison Operators

Operator	Meaning	Example
==	Equal to	5 == 5 -> True
!=	Not equal to	5 != 3 -> True
>	Greater than	7 > 3 -> True
<	Less than	2 < 9 -> True
>=	Greater than or equal to	5 >= 5 -> True
<=	Less than or equal to	4 <= 3 -> False

3.3 Logical Operators

Logical operators combine multiple boolean expressions into a single condition. and requires both sides to be true, or requires at least one side to be true, and not inverts a boolean value.

```

age = 20
has_id = True

if age >= 18 and has_id:
    print("Entry allowed")

is_weekend = False
is_holiday = True
if is_weekend or is_holiday:
    print("No work today")

print(not False)    # True

```

3.4 for Loops and range()

A for loop iterates over the elements of a sequence (a string, list, tuple, or the numbers produced by range()), executing the loop body once for every element. range(stop) produces numbers from 0 up to (but not including) stop. range(start, stop) starts from start instead of 0. range(start, stop, step) adds a custom step size, which can be negative to count downward.

```

for i in range(5):
    print(i)                # 0 1 2 3 4

for i in range(2, 10, 2):
    print(i)                # 2 4 6 8

for letter in "Python":
    print(letter)           # P y t h o n, one per line

for i in range(5, 0, -1):

```

```
print(i)                # 5 4 3 2 1
```

3.5 while Loops

A while loop repeats its body as long as its condition remains true. It is well suited to situations where the number of repetitions is not known ahead of time, such as repeatedly asking the user for input until they type something valid.

```
count = 0
while count < 5:
    print(count)
    count += 1      # without this, the loop would never end

# Input validation loop
answer = ""
while answer.lower() != "yes":
    answer = input("Type 'yes' to continue: ")
```

■ **Common Mistake:** a while loop whose condition never becomes false runs forever (an infinite loop). Always make sure something inside the loop body eventually makes the condition false.

3.6 break and continue

break immediately exits the nearest enclosing loop. continue skips the rest of the current iteration and jumps to the next one, without exiting the loop entirely.

```
for number in range(1, 10):
    if number == 6:
        break          # stop the loop entirely when we reach 6
    print(number)       # prints 1 2 3 4 5

for number in range(1, 6):
    if number % 2 == 0:
        continue      # skip even numbers
    print(number)      # prints 1 3 5
```

3.7 Nested Loops

A nested loop is a loop placed inside the body of another loop. The inner loop completes all of its iterations for every single iteration of the outer loop. Nested loops are common when working with grid-like or tabular data, such as printing a multiplication table.

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i * j}")
    print("---")
```

4. Syntax Summary

```
if condition:
    ...
elif other_condition:
    ...
else:
```

```

...

for item in iterable:
    ...

while condition:
    ...

range(stop)
range(start, stop)
range(start, stop, step)

break      # exit the loop
continue   # skip to next iteration

```

5. Code Examples

Example 1: Grade Classifier

```

score = int(input("Enter your score: "))
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
elif score >= 60:
    grade = "C"
else:
    grade = "F"
print(f"Your grade is {grade}")

```

Example 2: Sum of First N Natural Numbers

```

n = int(input("Enter n: "))
total = 0
for i in range(1, n + 1):
    total += i
print(f"Sum of first {n} numbers is {total}")

```

Example 3: Number Guessing Loop

```

secret = 7
guess = None
while guess != secret:
    guess = int(input("Guess the number (1-10): "))
    if guess < secret:
        print("Too low")
    elif guess > secret:
        print("Too high")
print("Correct!")

```

6. Step-by-Step Explanation

Walking through Example 1 (Grade Classifier) line by line:

- 1 The score is read from the user and converted to an integer.

- 2 Python checks the first condition, `score >= 90`; if true, grade is set to "A" and the remaining branches are skipped entirely.
- 3 If the first condition is false, Python checks the next `elif`, and so on, until one matches or the final `else` is reached.
- 4 Only one branch ever executes per run, because `elif` and `else` are mutually exclusive alternatives to the initial `if`.

7. Common Mistakes

■ **Common Mistake:** forgetting the colon (`:`) at the end of `if`, `for`, and `while` statements, causing a `SyntaxError`.

■ **Common Mistake:** writing an infinite `while` loop by forgetting to update the loop variable inside the body.

■ **Common Mistake:** using `=` instead of `==` inside a condition, which is a syntax error in Python (unlike some other languages) but still a frequent typo to watch for.

■ **Common Mistake:** assuming `range(1, 10)` includes 10 — it stops one before the given stop value.

■ **Common Mistake:** using `break` when `continue` was intended, accidentally exiting a loop early instead of just skipping one iteration.

8. Best Practices

✓ **Best Practice:** prefer `for` loops over `while` loops whenever you are iterating a known, fixed sequence, since they are less error-prone.

✓ **Best Practice:** keep conditions simple and readable; extract complex boolean expressions into well-named variables.

✓ **Best Practice:** avoid deeply nested loops (more than 2–3 levels) — consider refactoring into functions for clarity.

✓ **Best Practice:** always double-check the loop variable actually changes to avoid infinite loops.

9. Practice Questions

- 1 What does `range(3, 15, 4)` produce?
- 2 Rewrite `if not (a == b)` using a more Pythonic comparison operator.
- 3 What is the difference between `break` and `continue` in a `for` loop?
- 4 Trace through a `while` loop that starts at 10 and subtracts 3 each time until the value is less than or equal to 0 — list every value printed.
- 5 When would you choose a `while` loop over a `for` loop?

10. Mini Coding Exercises

- 1 Write a program that prints all even numbers from 1 to 50 using a for loop.
- 2 Write a program that checks if a number entered by the user is prime.
- 3 Write a program that prints the multiplication table of a number entered by the user.
- 4 Write a program using nested loops to print a right-angled triangle of stars.
- 5 Write a program that keeps asking the user for numbers and stops (using break) when they type 0, then prints the sum of all entered numbers.

11. Interview / QC Questions

- 1 Explain the difference between elif chains and multiple separate if statements.
- 2 What is short-circuit evaluation in the context of and / or operators?
- 3 How does Python's for loop differ conceptually from a C-style for loop?
- 4 What happens if you use break inside a nested loop — does it exit both loops or just the inner one?
- 5 Describe a real-world scenario where an infinite while loop with an internal break condition is the cleanest solution.

12. Summary

This module covered how Python programs make decisions with if/elif/else, how comparison and logical operators build the conditions those branches depend on, and how for and while loops repeat work — either over a known sequence via range() or until a condition changes. You also learned to control loop execution precisely with break and continue, and how nested loops let you work with multi-dimensional or tabular problems. Together, conditionals and loops are the core tools that let a program react to data and repeat work efficiently, and they will appear inside almost every function you write from here on.

End of Python_Control_Flow — continue to Python_Functions.pdf for the next module.